

Optimization Implementation Guide

Previous filename: fixes.md

Implementation guidelines derived from [architecture-analysis.md](#) and [optimization-report.md](#).

This document is the “how-to” companion to the optimization report. It contains actionable implementation patterns, code snippets, and configuration values that can be applied directly to the NovaDesk codebase.

Table of Contents

1. Auth Optimizations
2. Database Optimizations
3. Performance Optimizations
4. Security Hardening
5. Architecture Changes
6. Connection Pool Configuration
7. Module-Specific Fix Patterns
8. Testing & Validation

1. Auth Optimizations

1.1 Fire-and-Forget Email Dispatch

Problem: Nodemailer await blocks the HTTP response until the email leaves the server.

Implementation:

```
// utils/emailAsync.js
const sendEmailAsync = (emailFn, ...args) => {
  emailFn(...args).catch(err => console.error('Background email failed:', err));
};

module.exports = sendEmailAsync;
```

Usage in controllers:

```
// auth/controllers/auth_controller.js
import sendEmailAsync from '../../../utils/emailAsync.js';

// Registration
sendEmailAsync(emailService.sendOTP, email, otp, name, 'registration');

// OTP verification
sendEmailAsync(emailService.sendWelcomeEmail, email, name);

// Forgot password
sendEmailAsync(emailService.sendPasswordReset, email, resetToken);
```

Validation: Registration response time should drop from 3000–6000ms to 150–300ms.

1.2 bcrypt Rounds Reduction

Problem: bcrypt 12 rounds is too slow for high-concurrency auth.

Implementation:

```
// config/auth.config.js
module.exports = {
  BCrypt_Rounds: 8, // Reduced from 12
  // 8 rounds = ~80ms/hash (acceptable)
  // 12 rounds = ~300ms/hash (too slow under load)
};
```

Note: Only reduce if your threat model allows it. For NovaDesk's use case, 8 rounds is sufficient given JWT token expiry and rate limiting.

1.3 OTP Testing Toggle

Implementation:

```
// auth/models/otp.model.js
const OTP_DISABLED = process.env.NODE_ENV === 'development'; // or use explicit flag

if (OTP_DISABLED) {
  console.log('OTP DISABLED - Auto-verifying all OTPs');
  return { valid: true, testMode: true };
}
```

Important: Never set `OTP_DISABLED = true` in production. Use environment variables.

2. Database Optimizations

2.1 Index Creation Commands

Run these in MongoDB shell (mongosh) or via a migration script:

```
use novadesk-core;

// Auth
db.users.createIndex({ email: 1 }, { unique: true });
db.users.createIndex({ status: 1 });

// Storage
db.files.createIndex({ user: 1, folder: 1, isDeleted: 1, createdAt: -1 });
db.files.createIndex({ user: 1, isDeleted: 1, createdAt: -1 });
db.files.createIndex({ name: 'text' });
db.files.createIndex({ shareToken: 1, isDeleted: 1 });
```

```

db.folders.createIndex({ user: 1, parent: 1, isDeleted: 1 });
db.folders.createIndex({ user: 1, isStarred: 1, isDeleted: 1 });

// Team
db.teams.createIndex({ createdBy: 1, status: 1 });
db.teammembers.createIndex({ team: 1, user: 1 }, { unique: true });
db.teammembers.createIndex({ team: 1, status: 1 });
db.team_tasks.createIndex({ team: 1, status: 1 });
db.team_messages.createIndex({ team: 1, createdAt: -1 });

// Browser
db.browsers.createIndex({ user: 1 }, { unique: true });
db.browsers.createIndex({ username: 1 }, { sparse: true });

// Chat
db.chats.createIndex({ type: 1, participants: 1 });
db.chats.createIndex({ type: 1 });
db.messages.createIndex({ chat: 1, createdAt: -1 });

// Notifications
db.notifications.createIndex({ user: 1, read: 1, createdAt: -1 });
db.notificationsettings.createIndex({ user: 1 }, { unique: true });

// AI
db.aiagents.createIndex({ userRef: 1 }, { unique: true });

// Portfolio
db.portfolios.createIndex({ owner: 1 }, { unique: true, sparse: true });

// Admin/System
db.systems.createIndex({ key: 1 }, { unique: true });

```

2.2 Verify Index Usage

Always verify indexes are being used:

```

db.files.find({ user: ObjectId("..."), folder: ObjectId("..."), isDeleted: false })
  .explain("executionStats")

```

Look for "stage": "IXSCAN" — if you see "stage": "COLLSCAN", the index is not being used.

2.3 Query Optimization Patterns

Use `.lean()` for read-only queries:

```

// ❌ Before — returns full Mongoose documents
const files = await File.find({ user: userId });

// ✅ After — returns plain JSON (faster, less memory)
const files = await File.find({ user: userId }).lean();

```

Replace \$lookup with separate queries:

```
// ❌ Before — slow aggregation
const result = await File.aggregate([
  { $match: { user: userId } },
  { $lookup: { from: 'folders', localField: 'folder', foreignField: '_id', as: 'folderData' } }
]);

// ✅ After — two fast indexed queries
const files = await File.find({ user: userId }).lean();
const folderIds = [...new Set(files.map(f => f.folder.toString()))];
const folders = await Folder.find({ _id: { $in: folderIds } }).lean();
```

Batch queries instead of N+1:

```
// ❌ Before — N queries
const users = await User.find();
for (const user of users) {
  const profile = await Profile.findOne({ user: user._id }); // N queries!
}

// ✅ After — 2 queries
const users = await User.find().lean();
const userIds = users.map(u => u._id);
const profiles = await Profile.find({ user: { $in: userIds } }).lean();
const profileMap = new Map(profiles.map(p => [p.user.toString(), p]));
```

Use Promise.all() for independent writes:

```
// ❌ Before — sequential
await folder.save();
await storage.save();

// ✅ After — parallel
await Promise.all([folder.save(), storage.save()]);
```

Use bulkWrite() for cascading updates:

```
// ❌ Before — individual saves in a loop
for (const folder of foldersToUpdate) {
  folder.size += fileSize;
  await folder.save();
}

// ✅ After — single bulk operation
await Folder.bulkWrite(
  foldersToUpdate.map(folder => ({
    updateOne: {
      filter: { _id: folder._id },
      update: { $inc: { size: fileSize } }
    }
  }))
)
```

```
);
```

3. Performance Optimizations

3.1 Caching Implementation (Upstash Redis)

Setup:

```
// config/redis.config.js
import { Redis } from '@upstash/redis';

const redis = new Redis({
  url: process.env.UPSTASH_REDIS_REST_URL,
  token: process.env.UPSTASH_REDIS_REST_TOKEN,
});

// In-memory fallback
const memoryCache = new Map();

export const cache = {
  async get(key) {
    try {
      return await redis.get(key);
    } catch {
      return memoryCache.get(key) || null;
    }
  },

  async set(key, value, ttlSeconds = 600) {
    try {
      await redis.set(key, value, { ex: ttlSeconds });
    } catch {
      memoryCache.set(key, { value, expiry: Date.now() + ttlSeconds * 1000 });
    }
  },

  async del(key) {
    try {
      await redis.del(key);
    } catch {
      memoryCache.delete(key);
    }
    memoryCache.delete(key); // always clear fallback too
  }
};
```

Module-specific caching patterns:

```
// Storage module example
const CACHE_KEYS = {
```

```

files: (userId, folderId) => `files:${userId}:${folderId}`,
storage: (userId) => `storage:${userId}`,
folder: (folderId) => `folder:${folderId}`
};

// In controller/service
async function getFiles(userId, folderId) {
  const cacheKey = CACHE_KEYS.files(userId, folderId);
  const cached = await cache.get(cacheKey);
  if (cached) return cached;

  const files = await File.find({ user: userId, folder: folderId, isDeleted: false })
    .lean()
    .limit(50);

  await cache.set(cacheKey, files, 600); // 10 min TTL
  return files;
}

// Invalidate on write
async function uploadFile(userId, folderId, fileData) {
  const file = await File.create(fileData);
  await cache.del(CACHE_KEYS.files(userId, folderId));
  await cache.del(CACHE_KEYS.storage(userId));
  return file;
}

```

TTL Strategy:

Data Type	TTL	Rationale
File listings	10 min	Changes frequently on upload
Storage metadata	30 min	Changes less frequently
User profile	10 min	Changes on follow/unfollow
Team details	5 min	Changes on member add/ remove
Feed/home	5 min	Highly dynamic

3.2 Pagination

Always cap pagination:

```

// middleware/pagination.js
const paginate = (defaultLimit = 50, maxLimit = 100) => (req, res, next) => {
  const page = Math.max(1, parseInt(req.query.page) || 1);
  const limit = Math.min(maxLimit, parseInt(req.query.limit) || defaultLimit);
  req.pagination = { page, limit, skip: (page - 1) * limit };
  next();
};

```

```
module.exports = paginate;
```

Usage:

```
router.get('/files', authenticate, paginate(50, 100), fileController.list);
```

3.3 Query Timeouts

Prevent hanging queries:

```
// In Mongoose connection config
mongoose.connect(uri, {
  serverSelectionTimeoutMS: 5000,
  socketTimeoutMS: 10000,
});

// Per-query timeout
const files = await File.find({ user: userId })
  .maxTimeMS(10000) // 10 second timeout
  .lean();
```

4. Security Hardening

4.1 Rate Limiting Configuration

```
// middleware/rateLimiter.js
import rateLimit from 'express-rate-limit';

export const globalLimiter = rateLimit({
  windowMs: 60 * 1000, // 1 minute
  max: 100,
  keyGenerator: (req) => req.ip,
  handler: (req, res) => res.status(429).json({ error: 'Too many requests' }),
  validate: { ip: false } // required for express-rate-limit v7+
});

export const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 5,
  keyGenerator: (req) => req.body.email || req.ip,
  handler: (req, res) => res.status(429).json({ error: 'Too many login attempts' }),
  validate: { ip: false }
});

export const apiLimiter = (windowMs = 60 * 1000, max = 30) => rateLimit({
  windowMs,
  max,
  keyGenerator: (req) => req.user?._id?.toString() || req.ip,
```

```

    handler: (req, res) => res.status(429).json({ error: 'API rate limit exceeded' }),
    validate: { ip: false }
  });

export const ipLockout = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 5,
  keyGenerator: (req) => req.ip,
  handler: (req, res) => res.status(429).json({ error: 'IP temporarily locked' }),
  skipSuccessfulRequests: true, // only count failures
  validate: { ip: false }
});

```

Apply in app.js:

```

app.use('/api/v1/auth', ipLockout, authLimiter, authModule);
app.use('/api/v1/storage', apiLimiter(60_000, 30), storageModule);
app.use('/api/v1/teams', apiLimiter(60_000, 30), teamModule);
app.use('/api/v1/browser', apiLimiter(60_000, 30), browserModule);
app.use('/api/v1/notifications', apiLimiter(60_000, 30), notificationModule);

```

4.2 Health Check (No Auth)

```

// app.js — TOP of file, before any middleware
app.get('/api/v1/health', (req, res) => {
  res.status(200).json({
    status: 'healthy',
    timestamp: new Date().toISOString(),
    uptime: process.uptime(),
    version: process.env.npm_package_version
  });
});

```

5. Architecture Changes

5.1 Route Mounting (No Duplicates)

```

// app.js — CORRECT pattern
app.get('/api/v1/health', healthController.check);

app.use('/api/v1/auth', authModule);
app.use('/api/v1/storage', storageModule);
app.use('/api/v1/teams', teamModule);
app.use('/api/v1/browser', browserModule);
app.use('/api/v1/chat', chatModule);
app.use('/api/v1/notifications', notificationModule);
app.use('/api/v1/code', codeModule);
app.use('/api/v1/portfolio', portfolioModule);

```

```
app.use('/api/v1/admin', adminModule);
app.use('/api/v1/ai', aiModule);
```

Rules:

- Each module has ONE mount point
 - No bare /api/v1 mounts for modules with sub-routes
 - Health check is FIRST, before any middleware
-

5.2 Express Route Ordering

```
// browser/routes/browserRoutes.js — CORRECT order

// 1. Specific routes FIRST
router.get('/me', authBrowser, browserController.getMyProfile);
router.get('/followers', authBrowser, browserController.getFollowers);
router.get('/following', authBrowser, browserController.getFollowing);
router.get('/home/stats', authBrowser, browserController.getHomeStats);
router.get('/home/me', authBrowser, browserController.getHomeMe);

// 2. Parameterized routes SECOND
router.get('/:identifier', browserController.getPublicProfile);

// ❌ WRONG — catch-all before specific routes
// router.get('/:identifier', ...); // This would catch /me, /followers, etc.
// router.get('/me', ...);          // Never reached!
```

5.3 Node.js Cluster Mode

```
// cluster.js
import cluster from 'cluster';
import os from 'os';

const numCPUs = os.cpus().length;

if (cluster.isPrimary) {
  console.log(`Primary ${process.pid} starting ${numCPUs} workers...`);

  for (let i = 0; i < numCPUs; i++) {
    cluster.fork();
  }

  cluster.on('exit', (worker, code, signal) => {
    console.log(`Worker ${worker.process.pid} died. Restarting...`);
    cluster.fork();
  });
} else {
```

```
await import('./server.js');
console.log(`Worker ${process.pid} started`);
}
```

Run:

```
node cluster.js
# or with increased memory
node --max-old-space-size=4096 cluster.js
```

6. Connection Pool Configuration

6.1 MongoDB Connection

```
// config/database.config.js
const mongooseOptions = {
  // Connection pool
  maxPoolSize: 1000,      // Was 10 — increased for cluster mode
  minPoolSize: 200,       // Was 2 — maintain warm connections
  maxConnecting: 50,      // Max simultaneous new connections
  waitQueueTimeoutMS: 60000, // 1 minute queue timeout

  // Timeouts
  serverSelectionTimeoutMS: 5000,
  socketTimeoutMS: 10000,
  heartbeatFrequencyMS: 10000,

  // Retry
  retryWrites: true,
  retryReads: true,
};

mongoose.connect(process.env.MONGODB_URI, mongooseOptions);
```

6.2 Environment-Specific Sizing

Environment	maxPoolSize	minPoolSize	Rationale
Local dev	10	2	Single user, limited resources
Staging	100	20	Moderate testing load
Production (serverless)	10–20	2–5	Cold-start friendly
Production (long-running)	1000	200	Sustained high concurrency

⚠ Important: Serverless and long-running configurations are NOT interchangeable. Serverless functions should use smaller pools to avoid connection exhaustion. Long-running servers need large pools to handle sustained load.

7. Module-Specific Fix Patterns

7.1 Lazy Creation Pattern

Rule: Never auto-create on GET. Return null + needsInitialization flag.

```
// ❌ WRONG — auto-creates on read
const getProfile = async (userId) => {
  let profile = await Profile.findOne({ user: userId });
  if (!profile) {
    profile = await Profile.create({ user: userId }); // DON'T DO THIS
  }
  return profile;
};

// ✅ CORRECT — return null, lazy-create on write
const getProfile = async (userId) => {
  const profile = await Profile.findOne({ user: userId });
  if (!profile) {
    return { data: null, needsInitialization: true };
  }
  return { data: profile, needsInitialization: false };
};

const updateProfile = async (userId, updates) => {
  let profile = await Profile.findOne({ user: userId });
  if (!profile) {
    profile = await Profile.create({ user: userId, ...updates });
  } else {
    Object.assign(profile, updates);
    await profile.save();
  }
  return profile;
};
```

7.2 Empty State Handling

Rule: “No data” is not an error. Return 200 with empty data.

```
// ❌ WRONG
if (!member) {
  return res.status(404).json({ error: 'No team found' });
}

// ✅ CORRECT
if (!member) {
  return res.status(200).json({
    success: true,
    data: { memberCount: 0, taskCount: 0, channelCount: 0, messageCount: 0 },
    message: 'No team associated with this user'
  });
};
```

```
}
```

7.3 Model Import Safety

Rule: Never rely on side-effect imports. Import models directly.

```
// ❌ WRONG — relies on model being registered elsewhere
const TeamTask = mongoose.model('TeamTask'); // May throw MissingSchemaError

// ✅ CORRECT — explicit import
import TeamTask from '../models/team_task.model.js';
import TeamMessage from '../models/team_message_model.js';
```

7.4 Response Field Consistency

Rule: Always use data key for payload.

```
// ❌ WRONG — inconsistent field name
return res.json({ metadata: stats });

// ✅ CORRECT — matches all other endpoints
return res.json({ data: stats });
```

7.5 Middleware Scoping

Rule: Apply auth middleware only where needed. Don't use module-level middleware that auto-creates.

```
// ❌ WRONG — applies to ALL routes
router.use(authBrowser); // This runs on every route

// ✅ CORRECT — apply per-route
router.get('/me', authBrowser, getMyProfile);
router.get('/:identifier', getPublicProfile); // No authBrowser needed
```

8. Testing & Validation

8.1 Smoke Test (5 VUs)

```
k6 run tests/auth/auth-smoke-test.js
```

Expected: 100% success, P95 < 500ms

8.2 Stress Test (50–300 VUs)

```
k6 run --vus 50 --duration 1m tests/home/home-stress-test.js
```

Expected: > 95% success, P95 < 3s

8.3 Break Test (500–1000 VUs)

```
k6 run --vus 500 --duration 3m tests/combined/combined-stress-test.js
```

Expected: Identify failure threshold

8.4 Functional Check (PowerShell)

```
# auth-full-check.ps1
$body = @{ email="test@example.com"; password="Test@123456" } | ConvertTo-Json
$res = Invoke-RestMethod -Method POST -Uri "http://localhost:3800/api/v1/auth/login" -Body $body
    ↳ -ContentType "application/json"
$token = $res.data.tokens.accessToken
Invoke-RestMethod -Method GET -Uri "http://localhost:3800/api/v1/auth/me" -Headers @{
    ↳ Authorization="Bearer $token" }
```

8.5 Memory Monitoring

```
pm2 monit
# or
node --inspect cluster.js
```

Watch for:

- Memory per worker stable at 80–110MB
- No continuous growth (memory leak indicator)
- Event loop lag < 50ms

Related Documents

- [architecture-analysis.md](#) — The analysis that produced these guidelines
- [optimization-report.md](#) — Performance results after applying these fixes
- [scaling-test-data.md](#) — Raw test data validating these optimizations
- [lazy-loading-pattern.md](#) — Lazy-creation implementation across all modules